Algorítmica

2º Grado en Ingeniería Informática

Guion de prácticas

Análisis de Eficiencia de Algoritmos

Análisis de Eficiencia de Algoritmos

1. Objetivo

El objetivo de esta práctica es que el estudiante comprenda la importancia del análisis de la eficiencia de los algoritmos y se familiarice con las formas de llevarlo a cabo. Para ello se mostrará cómo realizar un estudio teórico, empírico e híbrido. Cada equipo de estudiantes deberá realizar los análisis empíricos e híbridos de los algoritmos que se detallan más adelante.

2. Cálculo de tiempo teórico

A partir de la expresión del algoritmo, se aplicarán las reglas conocidas para contar el número de operaciones que realiza un algoritmo. Este valor será expresado como una función T(n) que dará el número de operaciones requeridas para un caso concreto del problema caracterizado por tener un tamaño n.

En el caso de los algoritmos recursivos aparecerá una expresión del tiempo de ejecución con forma recursiva, que habrá que resolver con las técnicas estudiadas (p. ej. resolución de recurrencias por el método de la ecuación característica). El análisis que nos interesa será el del peor caso. Así, tras obtener la expresión analítica de T(n), calcularemos el orden de eficiencia del algoritmo empleando la notación O().

A continuación, desarrollaremos el estudio teórico sobre dos algoritmos de ejemplo.

2.1. Ejemplo 1: Algoritmo de ordenación Burbuja

Vamos a obtener la eficiencia teórica del algoritmo de ordenación burbuja. Para ello vamos a considerar el siguiente código que implementa la ordenación de un vector de enteros, desde la posición inicial a la final de este, mediante el método burbuja.

```
1   void burbuja(int T[], int inicial, int final) {
2       int i, j;
3       int aux;
4       for (i = inicial; i < final - 1; i++)
5           for (j = final - 1; j > i; j--)
6               if (T[j] < T[j-1]) {
7                   aux = T[j];
8                   T[j] = T[j-1];
9                   T[j-1] = aux;
10              }
11  }
```

La mayor parte del tiempo de ejecución se emplea en el cuerpo del bucle interno. Esta porción de código la podemos acotar por una constante a. Las líneas de 6-10 se ejecutan exactamente un número de veces igual a $(final - 1) - (i + 1) + 1$, es decir, $final - i - 1$. A su vez el bucle interno se ejecuta un número de veces indicado por el bucle externo. En definitiva, tendríamos una fórmula como la siguiente:

$$\sum_{i=inicial}^{final-2} \sum_{j=i+1}^{final-1} a \qquad (1)$$

Renombrando en la ecuación (1) *final* como *n* e *inicial* como 1, pasamos a resolver la siguiente ecuación:

$$\sum_{i=1}^{n-2} \sum_{j=i+1}^{n-1} a \qquad (2)$$

Realizando la sumatoria interior en (2) obtenemos:

$$\sum_{i=1}^{n-2} a(n - i - 1) \qquad (3)$$

Y finalmente tenemos:

$$\frac{a}{2}n^2 - \frac{3a}{2}n + a \qquad (4)$$

Claramente $\frac{a}{2}n^2 - \frac{3a}{2}n + a \in O(n^2)$. Diremos por tanto que el método de ordenación es de orden $O(n^2)$ o cuadrático.

2.2 Ejemplo 2: Algoritmo de ordenación Mergesort

En este ejemplo vamos a calcular la eficiencia del algoritmo de ordenación Mergesort. Este algoritmo divide el vector en dos partes iguales y se vuelve a aplicar de forma recurrente a cada una de ellas. Una vez hecho esto, fusiona los dos vectores sobre el vector original, de manera que esta parte ya queda ordenada. Si el número de elementos del vector que se está tratando en cada momento de la recursión es menor que una constante UMBRALMS, entonces se ordenará mediante el algoritmo burbuja.

El código del algoritmo MergeSort es el siguiente:

```
 1  void fusion(int T[], int inicial, int final, int U[], int V[])
 2  {
 3      int j = 0;
 4      int k = 0;
 5
 6      for (int i = inicial; i < final; i++)
 7          if (U[j] < V[k]) {
 8              T[i] = U[j];
 9              j++;
10          }
11          else {
12              T[i] = V[k];
13              k++;
14          }
15  }
16
```

```
17  void mergesort(int T[], int inicial, int final)
18  {
19      if (final - inicial < UMBRALMS)
20          burbuja(T, inicial, final);
21      else {
22          int k = (final - inicial) / 2;
23          int *U = new int[k - inicial + 1];
24          assert(U == 0);
25          int l, l2;
26
27          for (l = 0, l2 = inicial; l < k; l++, l2++)
28              U[l] = T[l2];
29
30          U[l] = INT_MAX;
31          int *V = new int[final - k + 1];
32          assert(V == 0);
33
34          for (l = 0, l2 = k; l < final - k; l++, l2++)
35              V[l] = T[l2];
36
37          V[l] = INT_MAX;
38          mergesort(U, 0, k);
39          mergesort(V, 0, final - k);
40          fusion(T, inicial, final, U, V);
41          delete[] U;
42          delete[] V;
43      }
44  }
```

Suponiendo que entramos por la rama else (línea 21) podemos acotar la secuencia de instrucciones de las líneas 22-25 por una constante. Sin perder generalidad, vamos a tomar esta constante como c. El bucle for de la línea 27 se ejecuta un número de veces igual a n/2 (en la primera llamada de mergesort tomaremos inicial como 0 y final como n), por lo tanto la eficiencia hasta aquí sería O(n). Este mismo razonamiento lo tenemos para las instrucciones desde la línea 30 hasta 37. Aplicando la regla del máximo obtendríamos hasta la línea 37 un orden de O(n).

A continuación, se hacen dos llamadas de forma recursiva a mergesort (líneas 38 y 39) con los dos nuevos vectores construidos para ordenar en cada uno de ellos n/2 elementos. En la línea 40 se llama al procedimiento fusión que como se puede observar tiene una eficiencia de O(n).

Por lo tanto, para averiguar la eficiencia de mergesort, podemos formular la siguiente ecuación[1]:

$$T(n) = \{2T\left(\frac{n}{2}\right) + n \; si \; n \geq UMBRALMS; \; O(1) \; si \; n < UMBRALMS\} \quad (5)$$

Vamos a resolver la recurrencia en la ec.(5):

$$T(n) = 2T\left(\frac{n}{2}\right) + n \quad si \; n \geq UMBRALMS \quad (6)$$

Haciendo en la ec.(6) el cambio de variable $n = 2^m$ obtenemos:

$$T(2^m) = 2T(2^{m-1}) + 2^m \quad si \; m \geq log_2(UMBRALMS) \quad (7)$$

$$T(2^m) - 2T(2^{m-1}) = 2^m \quad (8)$$

Si definimos $t_m = T(2^m)$, obtenemos:

[1] *Para n< UMBRALMS, se puede considerar constante en lugar de n^2 ya que el tamaño de entrada no influye.*

$$t_m - 2t_{m-1} = 2^m \quad (9)$$

La ecuación de recurrencia que se deduce de la ec.(9) es:

$$(x - 2) = 0 \quad (10)$$

La solución general de la recurrencia es:

$$t_m = c_1 2^m + c_2 m 2^m \quad (11)$$

Deshaciendo el cambio de variable ($2^m = n$):

$$T(n) = c_1 n + c_2 n log_2 n \quad (12)$$

De la ecuación (12) se deduce que la eficiencia de MergeSort pertenece a: $T(n) \in O(nlog_2(n))$

3. Cálculo de la eficiencia empírica

Veremos ahora cómo llevar a cabo un estudio puramente empírico del comportamiento de los algoritmos analizados. Para ello mediremos los recursos empleados (tiempo) para cada tamaño dado de las entradas (en el caso de los algoritmos de ordenación, el tamaño viene dado por el número de componentes del vector a ordenar).

3.1. Cálculo del tiempo de ejecución

Para obtener el tiempo empírico de una ejecución de un algoritmo definiremos en el código dos variables:

```
clock_t tantes;
clock_t tdespues;
```

En la variable `tantes` capturamos el valor del reloj antes de la ejecución del algoritmo al que queremos medir el tiempo. La variable `tdespues` contendrá el valor del reloj después de la ejecución del algoritmo en cuestión. Así, si deseamos obtener el tiempo del algoritmo de ordenación burbuja tendremos que poner algo parecido a lo siguiente:

```
tantes = clock();
burbuja(T,0,n);
tdespues = clock();
```

Para obtener el número de segundos simplemente emitiremos un mensaje como este:

```
cout << (double)(tdespues - tantes) / CLOCKS_PER_SEC << endl;
```

Con esta instrucción lo que hacemos es obtener la diferencia entre los dos instantes y pasarlo a segundos mediante la constante `CLOCKS_PER_SEC`. Para hacer uso de estas sentencias hay que usar la biblioteca `ctime`.

3.2. Problemas para tamaños de entrada pequeños

Para aquellos casos donde, por ejemplo, la cantidad de elementos a ordenar es muy pequeña, el tiempo medido será muy pequeño y por lo tanto el resultado será 0 segundos. Estos tiempos tan pequeños se pueden medir de forma indirecta ejecutando la sentencia que nos interesa muchas veces y después dividiendo el tiempo total por el número de veces que se ha ejecutado. Por ejemplo:

```
#include <ctime> ...
clock_t tantes,tdespues;
double tiempo_transcurrido;
const int NUM_VECES=10000;
int i;
tantes=clock();
for (i=0; i<NUM_VECES;i++)
//Sentencia cuyo tiempo se pretende medir
tdespues = clock();
tiempo_transcurrido=((double)(tdespues-tantes)/
                    (CLOCKS_PER_SEC* (double)NUM_VECES));
```

3.3. Otra forma más precisa de calcular el tiempo de ejecución

También se dispone de otra biblioteca de la STL, `chrono`, que permite calcular el tiempo de ejecución de forma algo más precisa (necesita C++ 11 o posterior). Para emplearla incluimos en el programa:

```
#include <chrono>
…
std::chrono::high_resolution_clock::time_point t0, tf;
std::chrono::duration<double> duracion;
```

Los instantes de tiempo, y la duración, los obtendremos de la siguiente forma:

```
t0= std::chrono::high_resolution_clock::now();
//Ejecución del algoritmo
tf= std::chrono::high_resolution_clock::now();

duracion = std::chrono::duration_cast<std::chrono::duration<double>>(tf -
t0);
```

Para otras formas de medir el tiempo, consultar, por ejemplo:
https://ichi.pro/es/8-formas-de-medir-el-tiempo-de-ejecucion-en-c-c-79591185633529

3.4. Cómo proceder

Para realizar un análisis de eficiencia empírica deberemos ejecutar el mismo algoritmo para diferentes tamaños de entrada.

Así, para un algoritmo de ordenación, por ejemplo, lo ejecutaremos para diferentes tamaños del vector a ordenar y obtendremos el tiempo (preferiblemente varias veces para cada tamaño, en cuyo caso obtendremos el tiempo medio o el tiempo máximo por tamaño[2]). Estos tiempos los almacenaremos en un fichero. Para simplificar las tareas, se puede usar una "macro", como la que se muestra a continuación:

```
#!/bin/csh
@ inicio = 100
@ fin = 20000
@ incremento = 200

set ejecutable = burbuja
set salida = burbuja.dat
@ i = $inicio
```

[2] *Esto tiene sentido en tanto en cuanto los tiempos de ejecución del algoritmo puedan variar considerablemente para entradas del mismo tamaño*

```
echo "" >> salida.dat
while ( $i < $fin)
   ./$ejecutable 42 $i >> $salida
   @ i += $incremento
end
```

Así en esta macro se va a escribir en el fichero salida.dat el tiempo en segundos que tarda el algoritmo de ordenación `burbuja` en ordenar vectores de 100 a 20000 elementos. Las muestras se han tomado de 200 en 200. Para poder ejecutarla debemos darle permisos de ejecución mediante la sentencia `chmod +x macro`, y a continuación ejecutarla como `./macro`.

3.5. Cómo mostrar los resultados

Para mostrar la eficiencia empírica haremos uso de tablas que recojan el tiempo invertido para cada caso y también de gráficas. Para mostrar los datos en gráfica pondremos en el eje X (abscisa) el tamaño de los casos y en el eje Y (ordenada) el tiempo, medido en segundos, requerido por la implementación del algoritmo. Para hacer esta representación de los resultados podemos hacer uso del software gnuplot, entre otras muchas opciones. Gnuplot está disponible para Linux, OS/2, MS Windows, OSX, VMS y muchas otras plataformas. A continuación se ilustra brevemente el uso de gnuplot en Linux.

Partimos de un conjunto de datos, por ejemplo `salida.dat` que contiene en cada fila pares de elementos (*x, y*) separados por espacios en blanco, como se indica a continuación. El primer elemento del par se corresponde con el tamaño del problema y el segundo elemento se corresponde con el tiempo.

```
100 1.9e-05
300 0.0001333
500 0.000342
700 0.0006605
900 0.0010912
1100 0.0015816
1300 0.0021703
....
```

Desde línea de comandos hacemos una llamada a gnuplot, y para poder representar estos puntos podemos ejecutar el comando

```
gnuplot> plot 'salida.dat' title 'Eficiencia' with points
```

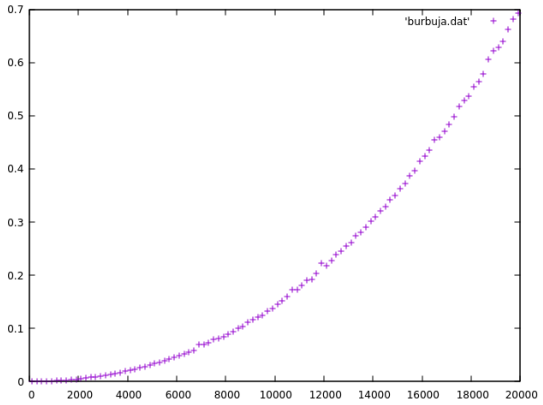

Figura 1: Fichero salida.dat

Como resultado aparecerá una nueva ventana como la de la figura 1. El comando plot indica que

queremos representar el fichero salida.dat, with points indica que en la salida nos muestre los datos como un conjunto de puntos desconectados (hay otras opciones, como por ejemplo with lines). Podemos dar un título significativo al grafico (en nuestro caso Eficiencia.)

Además podemos etiquetar los valores que representa el eje x y el eje y. Para ello podemos ejecutar los siguientes comandos:

```
gnuplot> set xlabel "Tamaño"
gnuplot> set ylabel "Tiempo (seg)"
```

Para volver a mostrar el gráfico, podemos utilizar el comando replot.

Para hacerse una idea de la potencia y calidad de los gráficos que puede generar gnuplot, se puede consultar la página web http://gnuplot.sourceforge.net/demo/.

4. Cálculo de la eficiencia híbrida

El cálculo teórico del tiempo de ejecución de un algoritmo nos da mucha información. Es suficiente para comparar dos algoritmos cuando los suponemos aplicados a casos de tamaño arbitrariamente grande. Sin embargo, cuando se va a aplicar el algoritmo en una situación concreta, es decir, especificadas la implementación, el compilador utilizado, el ordenador sobre el que se ejecuta, etc., nos interesa conocer de la forma más exacta posible la ecuación del tiempo. Así, el cálculo teórico nos da la expresión general, pero asociada a cada término de esta expresión aparece una constante de valor desconocido.

Para describir completamente la ecuación del tiempo, necesitamos conocer el valor de esas constantes. La forma de averiguar estos valores es ajustar la función a un conjunto de puntos. En nuestro caso, la función es la que resulta del cálculo teórico, el conjunto de puntos lo forman los resultados del análisis empírico y para el ajuste emplearemos regresión por mínimos cuadrados. Por ejemplo, en el algoritmo de ordenación burbuja la función que vamos a ajustar a los puntos obtenidos en el cálculo de la eficiencia empírica sería:

$$T(n) = a_0 \times n^2 + a_1 \times n + a_2$$

aunque también podríamos usar directamente

$$T(n) = a_0 \times n^2$$

Al ajustar a los puntos obtenidos por mínimos cuadrados obtendremos los valores de a_0, a_1 y a_2, es decir, las constantes ocultas. De esta manera, luego podremos saber cuánto tiempo aproximadamente utilizará el algoritmo para cualquier entrada de tamaño n.

4.1. Cómo obtener las constantes ocultas

Al igual que para el cálculo de la eficiencia empírica, podemos utilizar (entre otros) gnuplot. Ilustraremos a continuación el uso de gnuplot para esta tarea.

Lo primero que tenemos que hacer es definir la función que queremos ajustar a los datos. En nuestro ejemplo, estamos hablando de una función cuadrática, pues hemos visto que el algoritmo tiene un orden de eficiencia $O(n^2)$. Podemos definir esta función en gnuplot mediante el siguiente comando:

```
gnuplot> f(x) = a0*x*x+a1*x+a2
```

El siguiente paso es indicarle a gnuplot que haga la regresión. Esto es simple, únicamente le tenemos que indicar

```
gnuplot> fit f(x) 'salida.dat' via a0,a1,a2
```

Lo que se debe ver es el resultado, tras unas pocas iteraciones, del proceso de regresión de la función a ajustar. El ´ultimo paso debería ser algo como:

```
…
After 12 iterations the fit converged.
final sum of squares of residuals : 0.00320228
rel. change during last iteration : -2.03143e-15

degrees of freedom    (FIT_NDF)                        : 97
rms of residuals      (FIT_STDFIT) = sqrt(WSSR/ndf)    : 0.00574571
variance of residuals (reduced chisquare) = WSSR/ndf   : 3.30132e-05

Final set of parameters            Asymptotic Standard Error
=======================            =========================
a0              = 2.07535e-09      +/- 1.928e-11    (0.9288%)
a1              = -7.71649e-06     +/- 3.982e-07    (5.16%)
a2              = 0.013293         +/- 0.001724     (12.97%)

correlation matrix of the fit parameters:
                a0     a1     a2
a0              1.000
a1             -0.968  1.000
a2              0.745 -0.866  1.000
```

Como vemos, hay una gran cantidad de información sobre los datos y sobre la bondad del ajuste. Probablemente, la parte que más nos importa es cuáles son los valores de los parámetros. Estos son fácilmente identificables bajo la sección `Final set of parameters`.

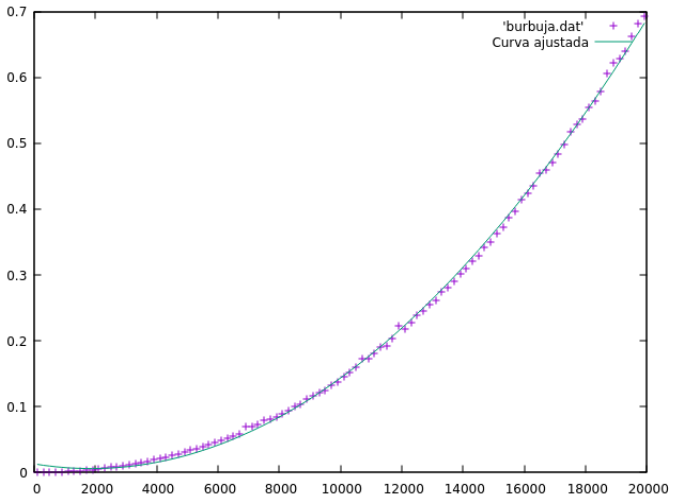

Figura 3: Enfoque híbrido: Ajuste de datos

La siguiente pregunta es cómo se ajusta esta función a nuestros datos. Para ello, podemos dibujar ambos en un único gráfico mediante el comando siguiente. Como resultado tenemos el gráfico de la figura 3.

```
gnuplot> plot 'salida.dat', f(x) title 'Curva ajustada'
```

5. Tareas a realizar

Hay que aplicar las pautas y conceptos descritos en las secciones anteriores para el análisis de un conjunto de 9 algoritmos. Los algoritmos considerados son los que se describen en la Tabla 1: inserción, selección, burbuja, mergeSort, heapSort, quickSort y shellSort son algoritmos de ordenación de vectores; Fibonacci: es un algoritmo para calcular los números de la sucesión de Fibonacci; Hanoi es un algoritmo que resuelve el famoso problema de las Torres de Hanoi.

Algoritmo	**Eficiencia**
Inserción, Selección, Burbuja, Shellsort[3]	$O(n^2)$
MergeSort, HeapSort	$O(n \log n)$
QuickSort	$\theta(n \log n)$; $O(n^2)$
Fibonacci	$O\left(\left(\frac{1+\sqrt{5}}{2}\right)^n\right)$ $\frac{1+\sqrt{5}}{2} \approx 1.618$
Hanoi	$O(2^n)$

Tabla 1: Algoritmos a emplear

Se deberán realizar las siguientes tareas:

1. Calculad la eficiencia empírica, siguiendo las indicaciones de la sección 3. Definir los tamaños de entrada según el orden de eficiencia siguiendo los siguientes criterios:
   - Orden $O(n^2)$: desde 8000 hasta 200000 con saltos de 8000.
   - Orden $O(n \log n)$: desde 500000 hasta 5000000 con saltos de 100000.
   - Orden $O\left(\left(\frac{1+\sqrt{5}}{2}\right)^n\right)$: desde 10 hasta 35 con saltos de 1.
   - Orden $O(2^n)$: desde 3 hasta 33 con saltos de 1.

   Generar tablas diferentes para los algoritmos de distinto orden de eficiencia (los algoritmos del mismo orden deben ejecutarse en el mismo ordenador): una con los algoritmos de orden $O(n^2)$, otra con los $O(n \log n)$ y otra con los exponenciales $O\left(\left(\frac{1+\sqrt{5}}{2}\right)^n\right)$ y $O(2^n)$.

2. Con cada una de las tablas anteriores, generad un gráfico comparando los tiempos de los algoritmos. Indicad claramente el significado de cada serie.

3. Calculad la eficiencia híbrida de los algoritmos, siguiendo las indicaciones de la sección 4. Utilizad ajustes al término dominante (p.e. en lugar de `f(x)=a0*x*x + a1*x + a2` use `f(x)=a0*x*x`. Probad también con otros ajustes que no se correspondan con la eficiencia teórica (p.e. un ajuste lineal a un algoritmo cuadrático, o un ajuste cúbico a un algoritmo exponencial, etc) y comprobad la variación en la calidad del ajuste.

4. Para los algoritmos de ordenación, incluid también una gráfica común con todos ellos, para poder apreciar las diferencias en rendimiento de algoritmos con diferente orden de eficiencia. Discuta dichas diferencias y valore la calidad de cada algoritmo de ordenación en términos de eficiencia. En este caso utilizad tamaños más pequeños para los algoritmos de tipo $O(n \log n)$ para poder visualizar mejor los resultados. Para el caso de Quicksort probad también con vectores que muestren el peor caso de dicho algoritmo. Para el caso de Heapsort y Shellsort probad también con cadenas que muestren el peor caso de los algoritmos.

[3] *Puede ser menor dependiendo de la secuencia de gaps que se utilice.*